

Assessment-2

Section A – Concept Application

Question 1

Scenario

You are a junior QA engineer at a food delivery startup. During sprint testing you log a critical defect: the order total is calculated incorrectly when a promo code is applied. The developer fixes the issue and marks it "**Fixed**" in JIRA. You pick it up for retesting, but the problem still persists.

Answer

Since the defect still exists after retesting, the bug should be moved to the **Reopened** state.

Next Three Steps in the Bug Life Cycle

Bug State	Responsible Person	Description
Reopened	QA Tester	The tester verifies the fix, finds the issue still exists, and reopens the defect.
Assigned	Test Lead / Project Manager	The reopened defect is reassigned to the appropriate developer for further investigation.
In Progress	Developer	The developer analyzes the root cause, modifies the code, and works on fixing the defect again.

Question 2

Scenario

You are testing a food delivery app and discover two defects:

- (a) The **Place Order** button becomes completely unresponsive after a coupon code is applied.
- (b) The company logo appears 4 pixels misaligned on the checkout screen.

Answer

Defect	Severity	Priority	Justification
(a) Place Order button not working	S1 (Critical)	P1 (High)	Users cannot complete orders, directly impacting business revenue and customer

Defect	Severity	Priority	Justification
			experience.
(b) Logo misaligned by 4 pixels	S4 (Low)	P3 (Low)	This is a cosmetic UI issue and does not affect application functionality.

Why can a defect have High Severity but Low Priority?

A defect may have **High Severity** because it significantly impacts the application's functionality, but **Low Priority** if it affects a rarely used feature or can be safely deferred.

Example: If the "Book Catering Order" feature (used by very few users) crashes, the defect has **High Severity** because the feature is unusable. However, if the business is currently focused on fixing critical issues in the regular food ordering flow, it may be assigned **Low Priority** until a later release.

Question 3

Scenario

The QA lead wants to automate regression testing for the search and filter functionality of a food delivery app using Selenium IDE.

Answer

Why is this a good candidate for automation?

1. Repetitive Testing

Regression testing is executed **twice every sprint**, making it repetitive. Automating these tests saves significant manual effort and reduces execution time.

2. Stable Functionality

Although the feature receives updates, the core search and filter workflow remains largely consistent. Stable, frequently executed tests are ideal candidates for automation.

Limitations of Selenium IDE

1. Limited Programming Capability

Selenium IDE does not support advanced programming logic such as complex loops, custom frameworks, or sophisticated data-driven testing.

2. Limited Cross-Browser and Integration Features

Although Selenium IDE supports major browsers, it is less flexible than Selenium WebDriver for advanced browser configurations, CI/CD integration, and large-scale automation projects.

Question 4

Scenario

The "Confirm Order" button has:

ID: btn-confirm-order

XPath: //div[@class='checkout-wrapper']/button[3]

The DOM structure changes frequently, but the ID remains stable.

Answer

The **ID locator** should be used.

Locator:

id=btn-confirm-order

Reason

An **ID** is unique, faster to locate, and remains stable across releases. Since the development team frequently changes the DOM structure, using XPath would make the test more vulnerable to failures.

What makes a locator fragile?

A locator is considered **fragile** when it depends on elements that change frequently, such as:

Absolute XPath

Element position

Dynamic class names

Deep DOM hierarchy

Consequence of using a fragile locator

If the checkout page layout changes during a sprint, the XPath may no longer locate the button even though the application works correctly. This results in **false test failures**, increasing maintenance effort and reducing confidence in the regression suite.

Question 5

Scenario

The Order History page displays:

- ✧ Customer Name
- ✧ Restaurant Name
- ✧ Order Amount

Database Tables:

users(user_id, name)

restaurants(restaurant_id, name)

orders(order_id, user_id, restaurant_id, amount, status)

Answer

JOIN Structure

```
SELECT
o.order_id,
u.name AS customer_name,
r.name AS restaurant_name,
o.amount,
o.status
FROM orders o
LEFT JOIN users u
ON o.user_id = u.user_id
LEFT JOIN restaurants r
ON o.restaurant_id = r.restaurant_id;
```

Why use LEFT JOIN instead of INNER JOIN?

A **LEFT JOIN** returns **all records from the Orders table**, even if matching user or restaurant records are missing.

Discrepancy Detected

Suppose an order exists in the Orders table, but its corresponding restaurant record was accidentally deleted.

INNER JOIN: The order disappears from the result set, hiding the problem.

LEFT JOIN: The order is still displayed, while the restaurant columns become **NULL**, clearly indicating a data integrity issue.

Therefore, LEFT JOIN is better for backend validation because it helps QA identify missing or inconsistent related data.

Question 6

Scenario

During payment testing, the user's wallet balance is debited, but no corresponding record is created in the Orders table.

Answer

ACID Property Violated

The violated property is **Atomicity**.

Reason

Atomicity requires that all operations in a transaction either complete successfully or none of them are applied.

In this case:

- Wallet balance was deducted.
- Order record was not created.

This indicates a **partial transaction**, violating Atomicity.

SQL Query 1

Verify wallet deduction.

```
SELECT
user_id,
wallet_balance
FROM users
WHERE user_id = 101;
```

This confirms whether the wallet amount was deducted.

SQL Query 2

Verify order creation.

```
SELECT *
FROM orders
WHERE user_id = 101
ORDER BY order_id DESC;
```

If no new order record exists while the wallet balance has decreased, the transaction failed partially.

Importance of ACID Compliance

ACID properties ensure database consistency and reliability.

Without Atomicity:

- ✧ Customers may lose money.
- ✧ Orders may not be recorded.
- ✧ Business reports become inaccurate.
- ✧ Customer trust decreases.
- ✧ Financial reconciliation becomes difficult.